

Source Code

1) Admin Login and Registration:

```
from logging import root
from tkinter import*
from tkinter import ttk
from PIL import Image,ImageTk
from tkinter import messagebox
import random
import time
import datetime
from time import strftime
import mysql.connector

def main():
    win=Tk()
    app=Login_Window(win)
    win.mainloop()

class Login_Window:
    def __init__(self,root):
        self.root=root
        self.root.title("Login Page")
        self.root.geometry("1550x800+0+0")
        img1 = Image.open("images/Library_Image.png")
        img1 = img1.resize((1530,800), Image.ANTIALIAS)

        self.photoImg1 = ImageTk.PhotoImage(img1)
        bg_lbl=Label(self.root,image=self.photoImg1)
```

```

bg_lbl.place(x=0,y=0,width=1530,height=800)

title=Label(bg_lbl,text="Library Management System",font=("times new
roman",42,"bold"),bg="orange",fg="red")
title.place(x=0,y=0,width=1550,height=70)

frame=Frame(self.root,bg="black")
frame.place(x=610,y=200,width=340,height=430)

img1=Image.open("images/LoginIconAppl.png")
img1=img1.resize((90,90),Image.ANTIALIAS)
self.photoimage1=ImageTk.PhotoImage(img1)
lblimg1=Label(image=self.photoimage1,bg="black",borderwidth=0)
lblimg1.place(x=730,y=200,width=90,height=90)

get_str=Label(frame,text="Admin Login",font=("times new
roman",20,"bold"),fg="white",bg="black")
get_str.place(x=95,y=85)

self.txtuser=StringVar()
self.txtpass=StringVar()

username=lbl=Label(frame,text="Username",font=("times new
roman",12,"bold"),fg="white",bg="black")
username.place(x=70,y=125)

txtuser=ttk.Entry(frame,textvariable=self.txtuser,font=("times new roman",15,"bold"))
txtuser.place(x=40,y=150,width=270)

```

```
password=lbl=Label(frame,text="Password",font=("times new  
roman",12,"bold"),fg="white",bg="black")  
password.place(x=70,y=195)
```

```
txtpass=ttk.Entry(frame,textvariable=self.txtpass,font=("times new  
roman",15,"bold"),show="*")  
txtpass.place(x=40,y=220,width=270)
```

```
img2=Image.open("images/LoginIconAppl.png")  
img2=img2.resize((25,25),Image.ANTIALIAS)  
self.photoimage2=ImageTk.PhotoImage(img2)  
lblimg1=Label(image=self.photoimage2,bg="black",borderwidth=0)  
lblimg1.place(x=650,y=323,width=25,height=25)
```

```
img3=Image.open("images/lock-512.png")  
img3=img3.resize((25,25),Image.ANTIALIAS)  
self.photoimage3=ImageTk.PhotoImage(img3)  
lblimg1=Label(image=self.photoimage3,bg="black",borderwidth=0)  
lblimg1.place(x=650,y=395,width=25,height=25)
```

```
btn_login=Button(frame,text="Login",borderwidth=3,relief=RAISED,command=self.login,c  
ursor="hand2",font=("times new roman",16,"bold"),fg="white",bg="red"  
,activebackground="#B00857")
```

```
btn_login.place(x=110,y=270,width=120,height=35)
```

```
registerbtn=Button(frame,text="New User  
Register",command=self.register_window,font=("times new
```

```
roman",10,"bold"),borderwidth=0,fg="white",bg="black",activeforeground="white",activebackground="black")
```

```
registerbtn.place(x=15,y=320,width=160)
```

```
forgetbtn=Button(frame,text="Forget  
Password",command=self.forgot_password_window,font=("times new  
roman",10,"bold"),borderwidth=0,fg="white",bg="black",activeforeground="white",activebackground="black")
```

```
forgetbtn.place(x=10,y=340,width=160)
```

```
def register_window(self):
```

```
self.new_window=Toplevel(self.root)
```

```
self.app=Register( self.new_window)
```

```
def login(self):
```

```
if self.txtuser.get()=="Pratik" and self.txtpass.get()=="Ketan":
```

```
    messagebox.showinfo("Success","Welcome to Library Management System...")
```

```
    self.new_window=Toplevel(self.root)
```

```
    self.app=LibraryManagementSystem(self.new_window)
```

```
elif self.txtuser.get()==" or self.txtpass.get()=="":
```

```
    messagebox.showerror("Error","all field required")
```

```
else:
```

```
conn=mysql.connector.connect(host="localhost",user="root",password="Ketan@16m",database="mydata")
```

```
my_cursor=conn.cursor()
```

```
my_cursor.execute("select * from register where email=%s and password=%s",(  
                                                            self.txtuser.get(),
```

```

self.txtpass.get()

))

row=my_cursor.fetchone()
if row==None:
    messagebox.showerror("Error","Invalid Username & password")
else:
    open_main=messagebox.askyesno("YesNo","Enter Library Management System")
    if open_main>0:
        pass
        self.new_window=Toplevel(self.root)
        self.app=LibraryManagementSystem(self.new_window)
    else:
        if not open_main:
            return
    conn.commit()
    self.clear()
    conn.close()
def clear(self):
    self.txtuser.set("")
    self.txtpass.set("")
def reset_pass(self):
    if self.combo_securiy_Q.get()=="Select" or self.txt_security.get()==" or
self.txt_newpass=="":
        messagebox.showerror("Error","All fields are required",parent=self.root2)
    else:
        try:

```

```

conn=mysql.connector.connect(host="localhost",user="root",password="Ketan@16m",database="mydata")

    cur=conn.cursor()
    query=("select * from register where email=%s and securityQ=%s and securityA=%s")
    value=(self.txtuser.get(),self.combo_securiy_Q.get(),self.txt_security.get(),)
    cur.execute(query,value)
    row=cur.fetchone()
    if row==None:
        messagebox.showerror("Error","Please select the correct security quetion/Enter answer",parent=self.root2)
    else:
        query=("update register set password=%s where email=%s")
        value=(self.txt_newpass.get(),self.txtuser.get())
        cur.execute(query,value)
        conn.commit()
        conn.close()
        messagebox.showinfo("Success","Your password has been reset,Please login with new password",parent=self.root2)
        self.root2.destroy()
        self.txtuser.focus()
    except Exception as es:
        messagebox.showerror("Error",f"Error Due To:{str(es)}",parent=self.root2)
def forgot_password_window(self):
    if self.txtuser.get()=="":
        messagebox.showerror("Error","Plaese Enter the Email address to reset password")
    else:

```

```

conn=mysql.connector.connect(host="localhost",user="root",password="Ketan@16m",database="mydata")

my_cursor=conn.cursor()
query=("select * from register where email=%s")
value=(self.txtuser.get(),)
my_cursor.execute(query,value)
row=my_cursor.fetchone()
if row==None:
    messagebox.showerror("My Error","Plaese enter the valid user name")
else:
    conn.close()
    self.root2=Toplevel()
    self.root2.title("Forget Password")
    self.root2.geometry("340x450+610+200")
    self.root2.configure(bg="white")
    l=Label(self.root2,text="Forget Password",font=("times new
roman",20,"bold"),fg="red",bg="white")
    l.place(x=0,y=10,relwidth=1)
    security_Q=Label(self.root2,text="Select Security Quetions",font=("times new
roman",15,"bold"),bg="white",fg="black")
    security_Q.place(x=50,y=80)
    self.combo_securiy_Q=ttk.Combobox(self.root2,font=("times new
roman",15,"bold"),state="readonly")
    self.combo_securiy_Q["values"]=("Select","Your Birth Place","Your Nick
name","Your Pet Name")
    self.combo_securiy_Q.place(x=50,y=110,width=250)
    self.combo_securiy_Q.current(0)

```

```

security_A=Label(self.root2,text="Security Answer",font=("times new
roman",15,"bold"),bg="white",fg="black")
security_A.place(x=50,y=150)
self.txt_security=ttk.Entry(self.root2,font=("times new roman",15,"bold"))
self.txt_security.place(x=50,y=180,width=250)
new_password=Label(self.root2,text="New Password",font=("times new
roman",15,"bold"),bg="white",fg="black")
new_password.place(x=50,y=220)
self.txt_newpass=ttk.Entry(self.root2,font=("times new roman",15,"bold"))
self.txt_newpass.place(x=50,y=250,width=250)
btn=Button(self.root2,text="Reset",command=self.reset_pass,font=("times new
roman",15,"bold"),fg="White",bg="green")
btn.place(x=120,y=290,width=100)

```

class Register:

```

def __init__(self,root):
    self.root=root
    self.root.title("Register")
    self.root.geometry("1600x900+0+0")

    self.var_fname=StringVar()
    self.var_lname=StringVar()
    self.var_contact=StringVar()
    self.var_email=StringVar()
    self.var_securityQ=StringVar()
    self.var_SecurityA=StringVar()
    self.var_pass=StringVar()
    self.var_confpass=StringVar()

```



```
self.bg=ImageTk.PhotoImage(file="images/employee_img2.jpg")
```

```
bg_lbl=Label(self.root,image=self.bg)
```

```
bg_lbl.place(x=0,y=0,relwidth=1,relheight=1)
```

```
self.bg1=ImageTk.PhotoImage(file="images/thought-good-morning-messages-  
LoveSove.jpg")
```

```
left_lbl=Label(self.root,image=self.bg1)
```

```
left_lbl.place(x=50,y=100,width=470,height=550)
```

```
frame=Frame(self.root,bg="white")
```

```
frame.place(x=520,y=100,width=800,height=550)
```

```
register_lbl=Label(frame,text="REGISTER HERE",font=("times new  
roman",20,"bold"),fg="darkgreen",bg="white")
```

```
fname=Label(frame,text="First Name",font=("times new  
roman",15,"bold"),bg="white")
```

```
fname.place(x=50,y=100)
```

```
self.fname_entry=ttk.Entry(frame,textvariable=self.var_fname,font=("times new  
roman",15,"bold"))
```

```
self.fname_entry.place(x=50,y=130,width=250)
```

```
l_name=Label(frame,text="Last Name",font=("times new  
roman",15,"bold"),bg="white",fg="black")
```

```
l_name.place(x=370,y=100)
```

```
self.txt_lname=ttk.Entry(frame,textvariable=self.var_lname,font=("times new  
roman",15,"bold"))
```

```

self.txt_lname.place(x=370,y=130,width=250)
contact=Label(frame,text="Contact No",font=("times new
roman",15,"bold"),bg="white",fg="black")
contact.place(x=50,y=170)
self.txt_contact=ttk.Entry(frame,textvariable=self.var_contact,font=("times new
roman",15,"bold"))
self.txt_contact.place(x=50,y=200,width=250)
email=Label(frame,text="Email",font=("times new
roman",15,"bold"),bg="white",fg="black")
email.place(x=370,y=170)
self.txt_email=ttk.Entry(frame,textvariable=self.var_email,font=("times new
roman",15,"bold"))
self.txt_email.place(x=370,y=200,width=250)
security_Q=Label(frame,text="Select Security Quetions",font=("times new
roman",15,"bold"),bg="white",fg="black")
security_Q.place(x=50,y=240)

self.combo_securiy_Q=ttk.Combobox(frame,textvariable=self.var_securityQ,font=("times
new roman",15,"bold"),state="readonly")
self.combo_securiy_Q["values"]=("Select","Your Birth Place","Your Nick name","Your
Blood Group")
self.combo_securiy_Q.place(x=50,y=270,width=250)
self.combo_securiy_Q.current(0)
security_A=Label(frame,text="Security Answer",font=("times new
roman",15,"bold"),bg="white",fg="black")
security_A.place(x=370,y=240)
self.txt_security=ttk.Entry(frame,textvariable=self.var_SecurityA,font=("times new
roman",15,"bold"))
self.txt_security.place(x=370,y=270,width=250)

```

```

pswd=Label(frame,text="Password ",font=("times new
roman",15,"bold"),bg="white",fg="black")
pswd.place(x=50,y=310)
self.txt_pswd=ttk.Entry(frame,textvariable=self.var_pass,font=("times new
roman",15,"bold"))
self.txt_pswd.place(x=50,y=340,width=250)
confirm_pswd=Label(frame,text="Confirm Password",font=("times new
roman",15,"bold"),bg="white",fg="black")
confirm_pswd.place(x=370,y=310)
self.txt_confirm_pswd=ttk.Entry(frame,textvariable=self.var_confpass,font=("times new
roman",15,"bold"))
self.txt_confirm_pswd.place(x=370,y=340,width=250)
self.var_check=IntVar()
self.checkbtn=Checkbutton(frame,variable=self.var_check,text="I Agree The Terms &
Conditions",bg='white',font=("times new roman",12,"bold"),onvalue=1,offvalue=0)
self.checkbtn.place(x=50,y=380)
img=Image.open("images/register-now-button1.jpg")
img=img.resize((200,55),Image.ANTIALIAS)
self.photoimage=ImageTk.PhotoImage(img)

b1=Button(frame,image=self.photoimage,command=self.register_data,borderwidth=0,cursor
="hand2",font=("times new roman",15,"bold"),fg="white")
b1.place(x=10,y=420,width=200)
img1=Image.open("images/loginpng.png")
img1=img1.resize((200,45),Image.ANTIALIAS)
self.photoimage1=ImageTk.PhotoImage(img1)

b1=Button(frame,image=self.photoimage1,command=self.return_login,borderwidth=0,cursor
="hand2",font=("times new roman",15,"bold"),fg="white")

```

```

b1.place(x=330,y=420,width=200)
def register_data(self):
    if self.var_fname.get()="" or self.var_email.get()="" or
self.var_securityQ.get()=="Select":
        messagebox.showerror("Error","All fields are required",parent=self.root)
    elif self.var_pass.get()!=self.var_confpass.get():
        messagebox.showerror("Error","password & confirm password must be
same",parent=self.root)
    elif self.var_check.get()==0:
        messagebox.showerror("Error","Plaese agree our terms ane
condition",parent=self.root)
    else:

conn=mysql.connector.connect(host="localhost",user="root",password="Ketan@16m",datab
ase="mydata")
    my_cursor=conn.cursor()
    query=("select * from register where email=%s")
    value=(self.var_email.get(),)
    my_cursor.execute(query,value)
    row=my_cursor.fetchone()
    if row!=None:
        messagebox.showerror("Error","User already exist,plaese try another
email",parent=self.root)
    else:
        my_cursor.execute("insert into register values(%s,%s,%s,%s,%s,%s,%s)",(
                                                                    self.var_fname.get(),
                                                                    self.var_lname.get(),
                                                                    self.var_contact.get(),
                                                                    self.var_email.get(),

```

```
        self.var_securityQ.get(),
        self.var_SecurityA.get(),
        self.var_pass.get()
    ))

    conn.commit()
    conn.close()
    messagebox.showinfo("Success", "Register Successfully")
def return_login(self):
    self.root.destroy()
```

2) Main Homescreen:

```
class LibraryManagementSystem:
```

```
    def __init__(self,root):
```

```
        self.root=root
```

```
        self.root.title("Library Management System")
```

```
        self.root.geometry("1550x800+0+0")
```

```
#=====Variables=====
```

```
    self.member_var=StringVar()
```

```
    self.ref_var=StringVar()
```

```
    self.title_var=StringVar()
```

```
    self.firstname_var=StringVar()
```

```
    self.lastname_var=StringVar()
```

```
    self.address1_var=StringVar()
```

```
    self.address2_var=StringVar()
```

```
    self.postcode_var=StringVar()
```

```
    self.mobile_var=StringVar()
```

```
    self.bookid_var=StringVar()
```

```
    self.booktitle_var=StringVar()
```

```
    self.auther_var=StringVar()
```

```
    self.dateborrowed_var=StringVar()
```

```
    self.datedue_var=StringVar()
```

```
    self.daysonbook=StringVar()
```

```
    self.lateratefine_var=StringVar()
```

```
    self.dateoverdue=StringVar()
```

```
    self.finallprice=StringVar()
```

```

#=====TitleLabel=====

lbltitle=Label(self.root,text="LIBRARY MANAGEMENT
SYSTEM",bg="white",fg="crimson",bd=20,relief=RIDGE,font=("times new
roman",50,"bold"),padx=2,pady=6)

lbltitle.pack(side=TOP,fill=X)

def time():
    string = strftime('%I:%M:%S %p')
    lbl.config(text = string)
    lbl.after(1000, time)

lbl = Label(lbltitle, font = ('times new roman',15, 'bold'),background =
'purple',foreground = 'white')

lbl.place(x=0,y=0,width=150)

time()

#=====Dataframe=====

DataFrame=Frame(self.root,bd=20,padx=20,relief=RIDGE)

DataFrame.place(x=0,y=130,width=1530,height=400)

DataFrameLeft=LabelFrame(DataFrame,bd=12,padx=20,relief=RIDGE,fg="darkgreen",
                        font=("arial",12,"bold"),text="Library Membership
Information")

DataFrameLeft.place(x=0,y=5,width=900,height=350)

DataFrameRight=LabelFrame(DataFrame,bd=12,padx=20,relief=RIDGE,fg="darkgreen",
                        font=("arial",12,"bold"),text="Book Details")

DataFrameRight.place(x=910,y=5,width=540,height=350)

#=====Buttonframe=====

ButtonFrame=Frame(self.root,bd=20,padx=20,relief=RIDGE)

```

```
ButtonFrame.place(x=0,y=530,width=1530,height=70)
```

```
# =====ButtonFrame=====
```

```
    btnAddData=Button(ButtonFrame,command=self.add_data,text="ADD  
DATA",font=("arial",12,"bold"),width=23,bg="crimson",fg="white")
```

```
    btnAddData.grid(row=0,column=0)
```

```
    btnShowData=Button(ButtonFrame,command=self.showData,text="SHOW  
DATA",font=("arial",12,"bold"),width=23,bg="crimson",fg="white")
```

```
    btnShowData.grid(row=0,column=1)
```

```
    btnUpdate=Button(ButtonFrame,command=self.update_data,text="UPDATE",font=("arial",12  
,"bold"),width=23,bg="crimson",fg="white")
```

```
    btnUpdate.grid(row=0,column=2)
```

```
    btnDelete=Button(ButtonFrame,command=self.mDelete,text="DELETE",font=("arial",12,"bol  
d"),width=23,bg="crimson",fg="white")
```

```
    btnDelete.grid(row=0,column=3)
```

```
    btnReset=Button(ButtonFrame,command=self.reset,text="RESET",font=("arial",12,"bold"),wi  
dth=23,bg="crimson",fg="white")
```

```
    btnReset.grid(row=0,column=4)
```

```
    btnExit=Button(ButtonFrame,command=self.iExit,text="EXIT",font=("arial",12,"bold"),width  
=23,bg="crimson",fg="white")
```



```
btnExit.grid(row=0,column=5)
```

```
#
```

```
=====Framedetails=====
```

```
=====
```

```
FrameDetails=Frame(self.root,bd=20,padx=20,relief=RIDGE)
```

```
FrameDetails.place(x=0,y=600,width=1530,height=195)
```

```
lblMember=Label(DataFrameLeft,font=("arial",12,"bold"),text="Member  
Type",padx=2,pady=6)
```

```
lblMember.grid(row=0,column=0,sticky=W)
```

```
comMember=ttk.Combobox(DataFrameLeft,textvariable=self.member_var,state="readonly",  
                        font=("arial",12,"bold"),width=27)
```

```
comMember['value']=("Admin Staf","Lecturer","Student")
```

```
comMember.current(0)
```

```
comMember.grid(row=0,column=1)
```

```
lblref=Label(DataFrameLeft,font=("arial",12,"bold"),text="PRN No:",padx=2)
```

```
lblref.grid(row=1,column=0,sticky=W)
```

```
txtref=Entry(DataFrameLeft,font=("arial",13,"bold"),textvariable=self.ref_var,width=29)
```

```
txtref.grid(row=1,column=1)
```

```
lblTitle=Label(DataFrameLeft,font=("arial",12,"bold"),text="ID No:",padx=2,pady=4)
```

```
lblTitle.grid(row=2,column=0,sticky=W)
```

```
txtTitle=Entry(DataFrameLeft,font=("arial",13,"bold"),textvariable=self.title_var,width=29)
```

```
txtTitle.grid(row=2,column=1)
```

```
lblFirstName=Label(DataFrameLeft,font=("arial",12,"bold"),text="FirstName:",padx=2,pady=6)
```

```
    lblFirstName.grid(row=3,column=0,sticky=W)
```

```
txtFirstName=Entry(DataFrameLeft,font=("arial",13,"bold"),textvariable=self.firstname_var,width=29)
```

```
    txtFirstName.grid(row=3,column=1)
```

```
lblLastName=Label(DataFrameLeft,font=("arial",12,"bold"),text="Surname:",padx=2,pady=6)
```

```
    lblLastName.grid(row=4,column=0,sticky=W)
```

```
txtLastName=Entry(DataFrameLeft,font=("arial",13,"bold"),textvariable=self.lastname_var,width=29)
```

```
    txtLastName.grid(row=4,column=1)
```

```
lblAddress1=Label(DataFrameLeft,font=("arial",12,"bold"),text="Address1:",padx=2,pady=6)
```

```
    lblAddress1.grid(row=5,column=0,sticky=W)
```

```
txtAddress1=Entry(DataFrameLeft,font=("arial",13,"bold"),textvariable=self.address1_var,width=29)
```

```
    txtAddress1.grid(row=5,column=1)
```

```
lblAddress2=Label(DataFrameLeft,font=("arial",12,"bold"),text="Address2:",padx=2,pady=6)
```

```
    lblAddress2.grid(row=6,column=0,sticky=W)
```

```
txtAddress2=Entry(DataFrameLeft,font=("arial",13,"bold"),textvariable=self.address2_var,width=29)
```

```
txtAddress2.grid(row=6,column=1)
```

```
lblPostCode=Label(DataFrameLeft,font=("arial",12,"bold"),text=" Post  
Code:",padx=2,pady=4)
```

```
lblPostCode.grid(row=7,column=0,sticky=W)
```

```
txtPostCode=Entry(DataFrameLeft,font=("arial",13,"bold"),textvariable=self.postcode_var,width=29)
```

```
txtPostCode.grid(row=7,column=1)
```

```
lblMobile=Label(DataFrameLeft,font=("arial",12,"bold"),text="Mobile  
Number:",padx=2,pady=6)
```

```
lblMobile.grid(row=8,column=0,sticky=W)
```

```
txtMobile=Entry(DataFrameLeft,font=("arial",13,"bold"),textvariable=self.mobile_var,width=29)
```

```
txtMobile.grid(row=8,column=1)
```

```
lblBookId=Label(DataFrameLeft,font=("arial",12,"bold"),text="Book Id:",padx=2)
```

```
lblBookId.grid(row=0,column=2,sticky=W)
```

```
txtBookId=Entry(DataFrameLeft,font=("arial",12,"bold"),textvariable=self.bookid_var,width=29)
```

```
txtBookId.grid(row=0,column=3)
```

```

    lblBookTitle=Label(DataFrameLeft,font=("arial",12,"bold"),text="Book
Title:",padx=2,pady=6)
    lblBookTitle.grid(row=1,column=2,sticky=W)

    txtBookTitle=Entry(DataFrameLeft,font=("arial",12,"bold"),textvariable=self.booktitle_var,wi
dth=29)
    txtBookTitle.grid(row=1,column=3)

    lblAuther=Label(DataFrameLeft,font=("arial",12,"bold"),text="Auther
Name:",padx=2,pady=6)
    lblAuther.grid(row=2,column=2,sticky=W)

    txtAuther=Entry(DataFrameLeft,font=("arial",12,"bold"),textvariable=self.auther_var,width=2
9)
    txtAuther.grid(row=2,column=3)

    lblDateBorrowed=Label(DataFrameLeft,font=("arial",12,"bold"),text="Date
Borrowed:",padx=2,pady=6)
    lblDateBorrowed.grid(row=3,column=2,sticky=W)

    txtDateBorrowed=Entry(DataFrameLeft,font=("arial",12,"bold"),textvariable=self.dateborrow
ed_var,width=29)
    txtDateBorrowed.grid(row=3,column=3,sticky=W)

    lblDateDue=Label(DataFrameLeft,font=("arial",12,"bold"),text="Date
Due:",padx=2,pady=6)
    lblDateDue.grid(row=4,column=2,sticky=W)

```

```
txtDateDue=Entry(DataFrameLeft,font=("arial",12,"bold"),textvariable=self.datedue_var,width=29)
```

```
txtDateDue.grid(row=4,column=3)
```

```
lblDaysOnBook=Label(DataFrameLeft,font=("arial",12,"bold"),text="Days On Book:",padx=2,pady=6)
```

```
lblDaysOnBook.grid(row=5,column=2,sticky=W)
```

```
txtDaysOnBook=Entry(DataFrameLeft,font=("arial",12,"bold"),textvariable=self.daysonbook,width=29)
```

```
txtDaysOnBook.grid(row=5,column=3)
```

```
lblLateReturnFine=Label(DataFrameLeft,font=("arial",12,"bold"),text="Late Return Fine:",padx=2,pady=6)
```

```
lblLateReturnFine.grid(row=6,column=2,sticky=W)
```

```
txtLateReturnFine=Entry(DataFrameLeft,font=("arial",12,"bold"),textvariable=self.lateratefine_var,width=29)
```

```
txtLateReturnFine.grid(row=6,column=3)
```

```
lblDateOverdate=Label(DataFrameLeft,font=("arial",12,"bold"),text="Date Over Due:",padx=2,pady=6)
```

```
lblDateOverdate.grid(row=7,column=2,sticky=W)
```

```
txtDateOverdate=Entry(DataFrameLeft,font=("arial",12,"bold"),textvariable=self.dateoverdue,width=29)
```

```
txtDateOverdate.grid(row=7,column=3)
```

```

lblActualPrice=Label(DataFrameLeft,font=("arial",12,"bold"),text="Actual
Price:",padx=2,pady=6)
lblActualPrice.grid(row=8,column=2,sticky=W)

txtActualPrice=Entry(DataFrameLeft,font=("arial",12,"bold"),textvariable=self.finallprice,wid
th=29)
txtActualPrice.grid(row=8,column=3)

#=====DataframeRight=====
#=====textBox=====

self.textBox=Text(DataFrameRight,font=("arial",12,"bold"),width=32,height=16,padx=2,pady=
6)
self.textBox.grid(row=0,column=2)

#=====ListBox=====

listScrollbar=Scrollbar(DataFrameRight)
listScrollbar.grid(row=0,column=1,sticky="ns")

ListOfBooks=['Head Firt Book','Learn Python The Hard Way','Python
Programming',"Secrete Rahshy",'Python CookBook','Into to Machine Learning','Fluent
Python','progrmming Python','The Algorithm','The tecnich Python',
'Machine tecno','My Python','Joss Ellif guru','Elite Jungle
python','Jungli Python','Mumbai Python','Pune Python','Guru Of Python','Yellow Dragan','Red
python',
'Machine python','Advance Python','Inton
Python','RedChilli Python','Ishq Python']

def SelectBook(event=""):
    value=str(bookList.get(bookList.curselection()))

```

```

x=value
if (x=="Head Firt Book"):
    self.bookid_var.set("BKID5487")
    self.booktitle_var.set("Python manual")
    self.auther_var.set("Paull berry")

    d1=datetime.date.today()
    d2=datetime.timedelta(days=15)
    d3=d1+d2
    self.dateborrowed_var.set(d1)
    self.datedue_var.set(d3)
    self.daysonbook.set("15")
    self.lateratefine_var.set("Rs.25")
    self.dateoverdue.set("NO")
    self.finallprice.set("Rs.375")

elif (x=="Learn Python The Hard Way"):
    self.bookid_var.set("BKID8796")
    self.booktitle_var.set("Basic Of Pythpn")
    self.auther_var.set("ZED A.SHAW")

    d1=datetime.date.today()
    d2=datetime.timedelta(days=15)
    d3=d1+d2
    self.dateborrowed_var.set(d1)
    self.datedue_var.set(d3)
    self.daysonbook.set("15")
    self.lateratefine_var.set("Rs.25")
    self.dateoverdue.set("NO")

```

```
self.finallprice.set("Rs.725")
```

```
elif (x=="Python Programming"):
```

```
self.bookid_var.set("BKID1245")
```

```
self.booktitle_var.set("Intro to python Comp Science")
```

```
self.auther_var.set("John Zhelle")
```

```
d1=datetime.date.today()
```

```
d2=datetime.timedelta(days=15)
```

```
d3=d1+d2
```

```
self.dateborrowed_var.set(d1)
```

```
self.datedue_var.set(d3)
```

```
self.daysonbook.set("15")
```

```
self.lateratefine_var.set("Rs.25")
```

```
self.dateoverdue.set("NO")
```

```
self.finallprice.set("Rs.500")
```

```
elif (x=="Secrete Rahshy"):
```

```
self.bookid_var.set("BKID8796")
```

```
self.booktitle_var.set("Basic Of Pythpn")
```

```
self.auther_var.set("Ref.Kapil Kamble")
```

```
d1=datetime.date.today()
```

```
d2=datetime.timedelta(days=15)
```

```
d3=d1+d2
```

```
self.dateborrowed_var.set(d1)
```

```
self.datedue_var.set(d3)
```



```
self.daysonbook.set("15")
self.lateratefine_var.set("Rs.25")
self.dateoverdue.set("NO")
self.finallprice.set("Rs.289")
```

```
elif (x=="Python CookBook"):
    self.bookid_var.set("BKID2546")
    self.booktitle_var.set("Python Cookbook")
    self.auther_var.set("Brian Jones")
```

```
d1=datetime.date.today()
d2=datetime.timedelta(days=15)
d3=d1+d2
self.dateborrowed_var.set(d1)
self.datedue_var.set(d3)
self.daysonbook.set("15")
self.lateratefine_var.set("Rs.25")
self.dateoverdue.set("NO")
self.finallprice.set("Rs.354")
```

```
elif (x=="Intro to Machine Learning"):
    self.bookid_var.set("BKID8796")
    self.booktitle_var.set("Intro to Machine Learning")
    self.auther_var.set("Sarah Guido")
```

```
d1=datetime.date.today()
d2=datetime.timedelta(days=15)
```

```

d3=d1+d2
self.dateborrowed_var.set(d1)
self.datedue_var.set(d3)
self.daysonbook.set("15")
self.lateratefine_var.set("Rs.25")
self.dateoverdue.set("NO")
self.finallprice.set("Rs.725")

```

```

bookList=Listbox(DataFrameRight,font=("arial",12,"bold"),width=20,height=16)
bookList.bind('<<ListboxSelect>>',SelectBook)
bookList.grid(row=0,column=0,padx=4)
listScrollbar.config(command=bookList.yview)

```

```

for item in ListOfBooks:
    bookList.insert(END,item)

```

```

#=====Scrollbar=====

```

```

Table_frame=Frame(FrameDetails,bd=6,relief=RIDGE)
Table_frame.place(x=0,y=1,width=1460,height=150)

```

```

scroll_x=ttk.Scrollbar(Table_frame,orient=HORIZONTAL)
scroll_y=ttk.Scrollbar(Table_frame,orient=VERTICAL)

```

```

self.library_table=ttk.Treeview(Table_frame,column=("member","ref","title","firtname","lastn
ame","adress1",

```

```

"adress2","postid","mobile","bookid","booktitle","auther","dateborrowed",

```

```

        "datedue","days","latereturnfine","dateoverdue","finalprice")
        ,xscrollcommand=scroll_x.set,yscrollcommand=scroll_y.set)
scroll_x.pack(side=BOTTOM,fill=X)
scroll_y.pack(side=RIGHT,fill=Y)

scroll_x.config(command=self.library_table.xview)
scroll_y.config(command=self.library_table.yview)

self.library_table.heading("member",text="Member Type")
self.library_table.heading("ref",text="Reference No.")
self.library_table.heading("title",text="Title")
self.library_table.heading("firstname",text="First Name")
self.library_table.heading("lastname",text="Last Name")
self.library_table.heading("adress1",text="Address1")
self.library_table.heading("adress2",text="Address2")
self.library_table.heading("postid",text="Post ID")
self.library_table.heading("mobile",text="Mobile Number")
self.library_table.heading("bookid",text="Book ID")
self.library_table.heading("booktitle",text="Book Title")
self.library_table.heading("auther",text="Auther")
self.library_table.heading("dateborrowed",text="Date Of Borrowed")
self.library_table.heading("datedue",text="Date Due")
self.library_table.heading("days",text="DaysOnBook")
self.library_table.heading("latereturnfine",text="LateReturnFine")
self.library_table.heading("dateoverdue",text="DateOverDue")
self.library_table.heading("finalprice",text="Final Price")

self.library_table["show"]="headings"

```

```

self.library_table.column("member",width=100)
self.library_table.column("ref",width=100)
self.library_table.column("title",width=100)
self.library_table.column("firtname",width=100)
self.library_table.column("lastname",width=100)
self.library_table.column("adress1",width=100)
self.library_table.column("adress2",width=100)
self.library_table.column("postid",width=100)
self.library_table.column("mobile",width=100)
self.library_table.column("bookid",width=100)
self.library_table.column("booktitle",width=100)
self.library_table.column("auther",width=100)
self.library_table.column("dateborrowed",width=100)
self.library_table.column("datedue",width=100)
self.library_table.column("days",width=100)
self.library_table.column("latereturnfine",width=100)
self.library_table.column("dateoverdue",width=100)
self.library_table.column("finalprice",width=100)
self.library_table.pack(fill=BOTH,expand=1)

self.fatch_data()
self.library_table.bind("<ButtonRelease-1>",self.get_cursor)
self.fatch_data()
# =====Function Declaration=====
def add_data(self):
if self.member_var.get()==" or self.postcode_var.get()=="":
    messagebox.showerror("Error","All Fields Are Required",parent=self.root)

else:

```

try:

```
conn=mysql.connector.connect(host="localhost",username="root",password="Ketan@16m",database="mydata")
```

```
    my_cursor=conn.cursor()
```

```
    my_cursor.execute("insert into library
```

```
values(%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s),(
```

```
    self.member_var.get(),
```

```
    self.ref_var.get(),
```

```
    self.title_var.get(),
```

```
    self.firstname_var.get(),
```

```
    self.lastname_var.get(),
```

```
    self.address1_var.get(),
```

```
    self.address2_var.get(),
```

```
    self.postcode_var.get(),
```

```
    self.mobile_var.get(),
```

```
    self.bookid_var.get(),
```

```
    self.booktitle_var.get(),
```

```
    self.auther_var.get(),
```

```
    self.dateborrowed_var.get(),
```

```
    self.datedue_var.get(),
```

```
    self.daysonbook.get(),
```

```
    self.lateratefine_var.get(),
```

```
    self.dateoverdue.get(),
```

```
    self.finallprice.get()
```

```
))
```

```
conn.commit()
```

```

        self.fetch_data()
        conn.close()
        messagebox.showinfo("Success","Member has been inserted",parent=self.root)

    except Exception as es:
        messagebox.showerror("Error",f" Must be enter Integer number,PRN NO&ID NO is
Primery Key :{str(es)}",parent=self.root)

def update_data(self):
    if self.ref_var.get()=="":
        messagebox.showerror("Error","All Fields Are Required",parent=self.root)
    else:

conn=mysql.connector.connect(host='localhost',username='root',password='Ketan@16m',data
base='mydata')
    my_cursor=conn.cursor()
    my_cursor.execute("update library set
Member_type=%s,ID_No=%s,FirstName=%s,LastName=%s,Address1=%s,Address2=%s,Pos
tCode=%s,Mobile=%s,Bookid=%s,Booktitle=%s,Auther=%s,DateBorrowed=%s,DateDue=%
s,DaysOfBook=%s,LateReturnFine=%s,DateOverDue=%s,FinalPrice=%s where
PRN_No=%s",

self.member_var.get(),

self.title_var.get(),

self.firstname_var.get(),

self.lastname_var.get(),

```

```
self.address1_var.get(),  
  
self.address2_var.get(),  
  
self.postcode_var.get(),  
  
self.mobile_var.get(),  
  
self.bookid_var.get(),  
  
self.booktitle_var.get(),  
  
self.auther_var.get(),  
  
self.dateborrowed_var.get(),  
  
self.datedue_var.get(),  
  
self.daysonbook.get(),  
  
self.lateratefine_var.get(),  
  
self.dateoverdue.get(),  
  
self.finallprice.get(),  
  
self.ref_var.get()
```

```
))
```

```
conn.commit()  
self.fatch_data()  
self.reset()
```

```

        conn.close()

        messagebox.showinfo("UPDATE","Record has been updated
successfully",parent=self.root)

    def fetch_data(self):

conn=mysql.connector.connect(host="localhost",username="root",password="Ketan@16m",d
atabase="mydata")

        my_cursor=conn.cursor()
        my_cursor.execute("select * from library")
        rows=my_cursor.fetchall()
        if len(rows)!=0:
            self.library_table.delete(*self.library_table.get_children())
            for i in rows:
                self.library_table.insert("",END,values=i)
            conn.commit()
        conn.close()

    def get_cursor(self,event=""):
        cursor_row=self.library_table.focus()
        content=self.library_table.item(cursor_row)
        row=content["values"]

        self.member_var.set(row[0]),
        self.ref_var.set(row[1]),
        self.title_var.set(row[2]),
        self.firstname_var.set(row[3]),
        self.lastname_var.set(row[4]),
        self.address1_var.set(row[5]),

```



```

self.address2_var.set(row[6]),
self.postcode_var.set(row[7]),
self.mobile_var.set(row[8]),
self.bookid_var.set(row[9]),
self.booktitle_var.set(row[10]),
self.auther_var.set(row[11]),
self.dateborrowed_var.set(row[12]),
self.datedue_var.set(row[13]),
self.daysonbook.set(row[14]),
self.lateratefine_var.set(row[15]),
self.dateoverdue.set(row[16]),
self.finallprice.set(row[17])

```

```

def mDelete(self):
if self.ref_var.get()=="":
    messagebox.showinfo("ERROR","First Select the Member!!",parent=self.root)
else:

```

```

conn=mysql.connector.connect(host='localhost',username='root',password='Ketan@16m',data
base='mydata')

```

```

my_cursor=conn.cursor()
query="delete from library where PRN_No=%s"
value=(self.ref_var.get(),)
my_cursor.execute(query,value)

```

```

conn.commit()
conn.close()
self.fatch_data()
self.reset()

```

```
messagebox.showinfo("DELETE","Member has been Deleted  
successfully",parent=self.root)
```

```
def iExit(self):  
    iExit=messagebox.askyesno("Library Management System","Confirm if you want to  
exit",parent=self.root)  
    if iExit>0:  
        self.root.destroy()  
        return
```

```
def reset(self):  
    self.member_var.set(""),  
    self.ref_var.set(""),  
    self.title_var.set(""),  
    self.firstname_var.set(""),  
    self.lastname_var.set(""),  
    self.address1_var.set(""),  
    self.address2_var.set(""),  
    self.postcode_var.set(""),  
    self.mobile_var.set(""),  
    self.bookid_var.set(""),  
    self.booktitle_var.set(""),  
    self.auther_var.set(""),  
    self.dateborrowed_var.set(""),  
    self.datedue_var.set(""),  
    self.daysonbook.set(""),  
    self.lateratefine_var.set(""),  
    self.dateoverdue.set(""),  
    self.finallprice.set("")
```

```
self.txtBox.delete("1.0",END)
```

```
def showData(self):
```

```
    self.txtBox.insert(END,"Member Type:\t\t"+ self.member_var.get() + "\n")
```

```
    self.txtBox.insert(END,"PRN No:\t\t"+ self.ref_var.get() + "\n")
```

```
    self.txtBox.insert(END,"ID No:\t\t"+ self.title_var.get() + "\n")
```

```
    self.txtBox.insert(END,"FirstName:\t\t"+ self.firstname_var.get() + "\n")
```

```
    self.txtBox.insert(END,"LastName:\t\t"+ self.lastname_var.get() + "\n")
```

```
    self.txtBox.insert(END,"Address1:\t\t"+ self.address1_var.get() + "\n")
```

```
    self.txtBox.insert(END,"Address2:\t\t"+ self.address2_var.get() + "\n")
```

```
    self.txtBox.insert(END,"Post Code:\t\t"+ self.postcode_var.get() + "\n")
```

```
    self.txtBox.insert(END,"Mobile No:\t\t"+ self.mobile_var.get() + "\n")
```

```
    self.txtBox.insert(END,"Book ID:\t\t"+ self.bookid_var.get() + "\n")
```

```
    self.txtBox.insert(END,"Book Title:\t\t"+ self.booktitle_var.get() + "\n")
```

```
    self.txtBox.insert(END,"Auther:\t\t"+ self.auther_var.get() + "\n")
```

```
    self.txtBox.insert(END,"DateBorrowed:\t\t"+ self.dateborrowed_var.get() + "\n")
```

```
    self.txtBox.insert(END,"DateDue:\t\t"+ self.datedue_var.get() + "\n")
```

```
    self.txtBox.insert(END,"DaysOnBook:\t\t"+ self.daysonbook.get() + "\n")
```

```
    self.txtBox.insert(END,"LateRateFine:\t\t"+ self.lateratefine_var.get() + "\n")
```

```
    self.txtBox.insert(END,"DateOverDue:\t\t"+ self.dateoverdue.get() + "\n")
```

```
    self.txtBox.insert(END,"FinallPrice:\t\t"+ self.finallprice.get() + "\n")
```

```
if __name__ == "__main__":
```

```
    main()
```